ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

BÁO CÁO GIỮA KÌ

TÌM HIỂU VỀ XML VÀ JSON

CƠ SỞ DỮ LIỆU WEB VÀ HỆ THỐNG THÔNG TIN
MAT3385

Giảng viên: PGS. TS. Vũ Tiến Dũng

Nhóm thực hiện:

Lường Tiến Dũng	24001657
Đồng Đức Dương	24001659
Nguyễn Quốc Doanh	24001656
Nguyễn Tùng Dương	24001661

Hà Nội - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

BÁO CÁO GIỮA KÌ

TÌM HIỂU VỀ XML VÀ JSON

CƠ SỞ DỮ LIỆU WEB VÀ HỆ THỐNG THÔNG TIN
MAT3385

Giảng viên: PGS. TS. Vũ Tiến Dũng

Nhóm thực hiện:

Lường Tiến Dũng	24001657
Đồng Đức Dương	24001659
Nguyễn Quốc Doanh	24001656
Nguyễn Tùng Dương	24001661

Hà Nội - 2026

PHÂN CÔNG NHIỆM VỤ

Công việc	**Người phụ trách**
Tìm tài liệu, tìm hiểu tổng quan đề tài XML và JSON	Tất cả thành viên
Điều phối, phân công, tổng hợp và hoàn thiện báo cáo	Lường Tiến Dũng
Thiết kế, lập trình ứng dụng demo và chuẩn bị kịch bản thao tác	Lường Tiến Dũng
Thiết kế, tổng hợp và hoàn thiện slide thuyết trình	Nguyễn Quốc Doanh
Nội dung XML: khái niệm, đặc điểm, cấu trúc, well-formed; DTD/XSD; XSLT	Đồng Đức Dương
Nội dung XPath/XQuery; JSON, parse/stringify; so sánh, lựa chọn XML/JSON và kết luận	Nguyễn Tùng Dương

MỤC LỤC

PHÂN CÔNG NHIỆM VỤ 1
LỜI NÓI ĐẦU 4
PHẦN 1. TỔNG QUAN VÀ CẤU TRÚC XML 5
1.1. Khái niệm và đặc điểm 5
1.2. Cấu trúc một tài liệu 5
1.3. Quy tắc well-formed 5
1.4. Cú pháp và quy tắc dữ liệu 6
1.5. Mô hình cây và cách đọc tài liệu sinh viên 6
1.6. Quy ước phần tử, thuộc tính và dữ liệu rỗng 6
1.7. Ký tự đặc biệt và bảo toàn nội dung 7
1.8. Tình huống lỗi và vị trí phát hiện 7
PHẦN 2. CÁC CÔNG NGHỆ LIÊN QUAN ĐẾN XML 9
2.1. DTD và XML Schema 9
2.2. XSLT 9
2.3. XPath và XQuery 10
2.4. Thiết kế DTD cho dữ liệu minh họa 10
2.5. Thiết kế và kiểm tra XSD 11
2.6. Ví dụ XSLT lọc và sắp xếp 11
2.7. Phân tích biểu thức XPath bằng ví dụ 12
2.8. XQuery tạo kết quả mới 13
PHẦN 3. JSON VÀ XỬ LÝ DỮ LIỆU TRONG JAVASCRIPT 14
3.1. Khái niệm và mô hình dữ liệu 14
3.2. JSON.parse và JSON.stringify 14
3.3. Kiểm tra cấu trúc sau khi parse 14
3.4. Phân biệt chuỗi JSON và object trong ứng dụng 15
3.5. Các lỗi JSON thường gặp khi thao tác 15
3.6. Sửa object và xuất lại dữ liệu 16
PHẦN 4. SO SÁNH VÀ LỰA CHỌN XML, JSON 17
4.1. Đối chiếu theo yêu cầu 17
4.2. Khi nào dùng XML? 17
4.3. Khi nào dùng JSON? 18
4.4. Vì sao không chuyển đổi XML và JSON một cách máy móc? 18
4.5. Ba tình huống lựa chọn định dạng 19
PHẦN 5. XÂY DỰNG ỨNG DỤNG MINH HỌA TRÊN TRÌNH DUYỆT 20
5.1. Mục tiêu và phạm vi 20
5.2. Luồng xử lý 20
5.3. Một số đoạn mã tiêu biểu 21
5.4. Mô hình dữ liệu trung gian và trách nhiệm của từng hàm 21
5.5. Phân tích quá trình kiểm tra tuổi và mã sinh viên 22
5.6. Luồng lỗi và việc giữ lại kết quả cũ 23
5.7. Tạo giao diện bằng DOM và xử lý dữ liệu nhập 23

5.8. Ma trận kiểm thử chức năng trình duyệt 23
5.9. Kiểm tra bổ sung cho các ví dụ công nghệ XML 25
5.10. Đánh giá kết quả và hướng hoàn thiện 26
KẾT LUẬN 27
TÀI LIỆU THAM KHẢO 27
PHỤ LỤC. BỘ VÍ DỤ VÀ HƯỚNG DẪN TÁI LẬP 29
A. Các tệp bàn giao 29
B. Chạy trang demo 29

LỜI NÓI ĐẦU

Trong các hệ thống Web, dữ liệu cần được biểu diễn theo một cấu trúc thống nhất để các thành phần có thể lưu trữ, trao đổi và xử lý. XML và JSON là hai định dạng văn bản thường được sử dụng cho mục đích này. Mặc dù cùng biểu diễn dữ liệu có cấu trúc, chúng có cú pháp, mô hình dữ liệu và hệ công nghệ đi kèm khác nhau.

Báo cáo tìm hiểu đặc điểm của XML, các quy tắc well-formed, vai trò của DTD/XSD, XSLT, XPath/XQuery; đồng thời trình bày cấu trúc JSON và hai thao tác parse, stringify trong JavaScript. Trên cơ sở đó, nhóm đối chiếu hai định dạng và xây dựng một trang demo cho phép thao tác trực tiếp trên cùng dữ liệu sinh viên.

Phạm vi báo cáo bám theo slide XML & JSON của nhóm và mã nguồn demo đi kèm. Ứng dụng minh họa chạy tại trình duyệt, không có máy chủ API hay cơ sở dữ liệu. Các công nghệ DTD, XSD, XSLT và XQuery được trình bày ở phần lý thuyết; ứng dụng hiện tại không triển khai các bộ xử lý này.

Nhóm mong nhận được góp ý của giảng viên và các bạn để hoàn thiện cách trình bày cũng như ứng dụng minh họa.

PHẦN 1. TỔNG QUAN VÀ CẤU TRÚC XML

1.1. Khái niệm và đặc điểm

XML (Extensible Markup Language) là ngôn ngữ đánh dấu cho phép người viết đặt tên phần tử để mô tả dữ liệu. Tài liệu gồm các phần tử có thể lồng nhau, tạo thành cấu trúc cây. Tên thẻ giúp người đọc nhận biết ý nghĩa, nhưng phần mềm vẫn cần biết quy ước của tài liệu để xử lý đúng. XML mô tả dữ liệu; cách hiển thị được quyết định bởi phần mềm hoặc bước biến đổi riêng. [1]

XML sử dụng văn bản và không gắn với một ngôn ngữ lập trình cụ thể. Đặc điểm này thuận tiện cho trao đổi dữ liệu giữa các thành phần khác nhau. Tuy nhiên, hai hệ thống vẫn phải thống nhất tên trường, cấu trúc và ý nghĩa của dữ liệu; sử dụng XML không tự động giải quyết mọi vấn đề tương thích.

1.2. Cấu trúc một tài liệu

Ví dụ sau sử dụng cùng mô hình sinh viên như ứng dụng minh họa:

```
<?xml version="1.0" encoding="UTF-8"?>
<danhSachSinhVien>
  <sinhVien id="SV001">
    <hoTen>Lường Tiến Dũng</hoTen>
    <tuoi>20</tuoi>
    <email>dung@example.com</email>
  </sinhVien>
</danhSachSinhVien>
```

Trong ví dụ, danhSachSinhVien là phần tử gốc; sinhVien là phần tử con; id là thuộc tính. hoTen, tuoi và email chứa nội dung văn bản. Khai báo XML ở đầu nêu phiên bản và cách mã hóa; khai báo này không phải một phần tử dữ liệu. Tuổi và email trong ví dụ chỉ là dữ liệu minh họa, không phải thông tin xác thực của thành viên nhóm.

1.3. Quy tắc well-formed

Một tài liệu XML phải có đúng một phần tử gốc, các thẻ đóng/mở khớp nhau và lồng đúng thứ tự. Tên phần tử phân biệt chữ hoa, chữ thường; giá trị thuộc tính phải nằm trong dấu nháy. Các ký tự đặc biệt cũng cần được biểu diễn đúng, chẳng hạn dấu & trong nội dung phải viết thành &. [1]

Ví dụ <a> lồng đúng, trong khi <a> sai. Trong demo, nút Tạo lỗi thử đổi thẻ đóng hoTen thành ten để người xem quan sát phản hồi khi phân tích XML.

1.4. Cú pháp và quy tắc dữ liệu

Well-formed phản ánh cú pháp XML. Một tài liệu đúng cú pháp vẫn có thể thiếu trường bắt buộc hoặc chứa tuổi không phù hợp với yêu cầu ứng dụng. Để kiểm tra các yêu cầu đó, cần thêm quy tắc về cấu trúc và dữ liệu. Trong hệ công nghệ XML, DTD và XSD có thể được dùng để mô tả các ràng buộc; trong demo hiện tại, nhóm thực hiện kiểm tra riêng bằng JavaScript.

1.5. Mô hình cây và cách đọc tài liệu sinh viên

Có thể đọc mô hình dữ liệu của nhóm theo ba mức: danh sách, từng sinh viên và từng trường dữ liệu. Cấp danh sách cho phép gom nhiều đối tượng vào cùng tài liệu; cấp sinh viên thể hiện một bản ghi; các trường hoTen, tuoi, email mô tả giá trị của bản ghi đó. Khi thêm một sinh viên, nhóm thêm một nhánh sinhVien cùng cấp với các nhánh đã có, không tạo thêm phần tử gốc.

```
danhSachSinhVien
  +-- sinhVien [id=SV001]
  |     +-- hoTen: Lường Tiến Dũng
  |     +-- tuoi: 20
  |     +-- email: dung@example.com
  +-- sinhVien [id=SV002]
        +-- hoTen: Đồng Đức Dương
        +-- tuoi: 19
        +-- email: ducduong@example.com
```

Trong DOM, xuống dòng và khoảng trắng giữa các thẻ cũng có thể tạo node văn bản. Do đó, mã demo duyệt children khi cần các phần tử con, thay vì coi mọi childNodes là trường dữ liệu. Nếu không phân biệt, thao tác đếm trường có thể tính cả khoảng trắng do người viết thụt lề. Đây là một điểm quan trọng khi chuyển từ việc nhìn tài liệu bằng mắt sang xử lý bằng chương trình.

1.6. Quy ước phần tử, thuộc tính và dữ liệu rỗng

Nhóm chọn id làm thuộc tính vì nó định danh bản ghi; các thông tin còn lại là phần tử con. Có thể thiết kế một tài liệu khác đặt id thành phần tử, nhưng đó là một mô hình khác và parser cần được điều chỉnh. XML không tự quyết định mọi thông tin

định danh phải là thuộc tính. Yêu cầu quan trọng hơn là mô hình được thống nhất giữa nơi sinh dữ liệu và nơi đọc dữ liệu.

Ví dụ <email/> và <email></email> đều có thể biểu diễn phần tử không chứa nội dung. Trong ứng dụng của nhóm, cả hai không đạt quy tắc email phải là chuỗi không rỗng. Ngược lại, thiếu hẳn email là lỗi cấu trúc vì không có trường được yêu cầu. Phân biệt rỗng và thiếu giúp thiết kế phản hồi chính xác hơn khi kiểm tra dữ liệu.

Demo chỉ kiểm tra email là chuỗi không rỗng, chưa kiểm tra cú pháp địa chỉ thư điện tử hay khả năng nhận thư. Vì vậy, giá trị "abc" vẫn có thể qua bước kiểm tra email hiện tại. Đây là giới hạn cụ thể của mô hình, cần được nêu rõ thay vì mô tả ứng dụng như một hệ thống xác minh thông tin sinh viên.

1.7. Ký tự đặc biệt và bảo toàn nội dung

Giả sử tên minh họa là A & B. Khi xuất XML, ký tự & phải được viết thành & để không bị hiểu như mở đầu một tham chiếu thực thể. Tương tự, dấu < trong văn bản phải được escape; khi ghi vào thuộc tính nằm trong dấu nháy kép, dấu nháy kép cũng phải được xử lý. Hàm escapeXml trong app.js thực hiện bước này trước khi ghép chuỗi đầu ra.

```
<sinhVien id="SV005">
  <hoTen>A &amp;amp; B</hoTen>
  <tuoi>20</tuoi>
  <email>demo@example.com</email>
</sinhVien>
```

Sau khi phân tích XML, textContent của hoTen là A & B; người dùng không cần xử lý lại chuỗi & trong dữ liệu đã được parser giải mã. Nếu escape lần nữa ở sai thời điểm, có thể tạo tình trạng mã hóa hai lần. Trong demo, chỉ escape tại bước xuất XML, còn dữ liệu trung gian được giữ dưới dạng chuỗi thông thường.

1.8. Tình huống lỗi và vị trí phát hiện

Thay đổi dữ liệu	**Loại vấn đề**	**Bước phát hiện trong demo**
Đóng hoTen bằng thẻ ten	Sai cú pháp	DOMParser và parsererror

Thay đổi dữ liệu	**Loại vấn đề**	**Bước phát hiện trong demo**
Đổi root thành students	Sai mô hình demo	Kiểm tra tên phần tử gốc
Đưa email trước tuoi	Sai thứ tự mô hình	So sánh danh sách phần tử con
Tuổi là chữ hai mươi	Sai giá trị theo yêu cầu	Kiểm tra chuỗi tuổi trước khi ép số
Hai sinh viên cùng id	Vi phạm định danh	Tập Set trong validate

Từ bảng trên có thể thấy một thông báo chung "XML sai" chưa đủ để giải thích bản chất lỗi. Nhóm phân tách lỗi parser và lỗi quy tắc để người xem hiểu XML vẫn có thể đúng cú pháp nhưng không dùng được cho một ứng dụng cụ thể.

PHẦN 2. CÁC CÔNG NGHỆ LIÊN QUAN ĐẾN XML

2.1. DTD và XML Schema

DTD (Document Type Definition) mô tả các phần tử, thuộc tính, thứ tự và số lần xuất hiện được phép. XML Schema (XSD) cũng mô tả cấu trúc nhưng bổ sung hệ kiểu dữ liệu và các ràng buộc như khoảng giá trị, độ dài hoặc mẫu ký tự. XSD phù hợp khi cần kiểm tra chi tiết nội dung dữ liệu, chẳng hạn tuổi là số nguyên không âm. [1][3]

Tiêu chí	DTD	XSD
Quy định cấu trúc phần tử	Có	Có
Kiểu dữ liệu	Khả năng hạn chế	Có hệ kiểu dữ liệu phong phú
Cú pháp mô tả	Cú pháp DTD riêng	Cú pháp XML
Ràng buộc giá trị	Hạn chế	Có thể mô tả nhiều loại ràng buộc

Việc chọn DTD hay XSD phụ thuộc yêu cầu và hệ thống sẵn có. Không nên kết luận một công nghệ luôn cần thiết cho mọi tài liệu XML. Đối với demo nhỏ, kiểm tra cấu trúc bằng JavaScript giúp minh họa sự khác nhau giữa cú pháp và yêu cầu ứng dụng, nhưng không được gọi là kiểm định XSD.

2.2. XSLT

XSLT là ngôn ngữ biến đổi XML thông qua stylesheet. Một ứng dụng điển hình là nhận danh sách sinh viên ở XML và tạo bảng HTML. Quy tắc biến đổi có thể chọn trường, sắp xếp hoặc lọc dữ liệu; XPath thường được dùng để xác định các node cần xử lý. XSLT không đồng nghĩa với CSS: một bên biến đổi cấu trúc đầu ra, một bên chủ yếu mô tả cách trình bày. [4]

Trong demo, bảng HTML được tạo bằng JavaScript và DOM, không phải XSLT. Nếu mở rộng đề tài, nhóm có thể thêm stylesheet XSLT để so sánh hai cách tạo cùng một bảng dữ liệu.

2.3. XPath và XQuery

XPath mô tả cách chọn hoặc tính toán trên các node của tài liệu XML. Ví dụ, biểu thức /danhSachSinhVien/sinhVien/hoTen lấy tên sinh viên; biểu thức dưới đây bổ sung điều kiện tuổi:

```
/danhSachSinhVien/sinhVien[tuoi>=20]/hoTen
```

Biểu thức count(/danhSachSinhVien/sinhVien) trả số lượng phần tử sinhVien. Ứng dụng dùng Document.evaluate() của trình duyệt để thực hiện những truy vấn này và hiển thị kết quả. Các ví dụ dùng cú pháp XPath cơ bản được API này hỗ trợ. [5][6]

XQuery hướng tới truy vấn và tạo kết quả từ dữ liệu XML, hỗ trợ các thao tác như lặp, lọc, sắp xếp và xây dựng kết quả mới. XQuery có liên hệ với XPath nhưng không thể xem hai tên gọi là một công nghệ duy nhất. Phần này thuộc nội dung tìm hiểu; demo không có bộ thực thi XQuery. [7]

2.4. Thiết kế DTD cho dữ liệu minh họa

Tệp sinh-vien.dtd trong thư mục report/examples là ví dụ do nhóm xây dựng cho mô hình sinh viên. Nội dung sau cho phép danh sách có từ không đến nhiều sinh viên, mỗi sinh viên chứa đúng ba trường theo thứ tự và bắt buộc có mã định danh.

```
<!ELEMENT danhSachSinhVien (sinhVien*)>
<!ELEMENT sinhVien (hoTen,tuoi,email)>
<!ATTLIST sinhVien id ID #REQUIRED>
<!ELEMENT hoTen (#PCDATA)>
<!ELEMENT tuoi (#PCDATA)>
<!ELEMENT email (#PCDATA)>
```

Dấu * ở sinhVien* là lựa chọn của nhóm nhằm cho phép danh sách rỗng, tương ứng cách ứng dụng chấp nhận mảng rỗng. Khai báo id kiểu ID được dùng để ràng buộc định danh trong ví dụ DTD. Vì tuoi chỉ được khai báo là nội dung ký tự, tài liệu chứa <tuoi>hai muoi</tuoi> vẫn vượt qua DTD này. Như vậy, kết quả DTD hợp lệ không chứng minh tài liệu đáp ứng toàn bộ yêu cầu về tuổi.

Ví dụ DTD được kiểm tra bằng chương trình Python riêng. Không đưa DOCTYPE vào ô XML của trang web để chạy thử DTD, vì phiên bản demo hiện tại chủ động

từ chối khai báo đó. Phân biệt môi trường chạy giúp tránh trình diễn một công nghệ mà ứng dụng chưa thực hiện.

2.5. Thiết kế và kiểm tra XSD

Tệp sinh-vien.xsd bổ sung kiểu tuổi và tính duy nhất của mã. Phần khai báo dưới đây là trích đoạn từ tệp ví dụ, không phải một schema độc lập hoàn chỉnh:

```
<xs:sequence>
  <xs:element name="hoTen" type="ChuoiKhongRong"/>
  <xs:element name="tuoi" type="xs:nonNegativeInteger"/>
  <xs:element name="email" type="ChuoiKhongRong"/>
</xs:sequence>
<xs:attribute name="id" type="ChuoiKhongRong"
              use="required"/>
```

Trong schema đầy đủ, ChuoiKhongRong là kiểu riêng của ví dụ, xây dựng từ xs:token với minLength bằng 1. Root chứa chuỗi sinhVien với minOccurs bằng 0 và maxOccurs bằng unbounded. Ràng buộc xs:unique dùng selector sinhVien và field @id để phát hiện mã trùng. Những lựa chọn này làm rõ mô hình đang kiểm tra, thay vì chỉ liệt kê tên các kiểu dữ liệu của XSD.

Ví dụ XSD và validate trong JavaScript có mục tiêu gần nhau nhưng không hoàn toàn tương đương. XSD sử dụng kiểu nonNegativeInteger không bị giới hạn giống Number của JavaScript; demo yêu cầu Number.isSafeInteger để tránh dùng số vượt miền số nguyên an toàn. Schema cũng có quy tắc riêng về cấu trúc, còn parser JavaScript hiện tại không từ chối mọi thuộc tính ngoài mô hình. Vì vậy, không nên dùng kết quả của một bộ kiểm tra để thay thế kết quả bộ còn lại trong mọi trường hợp.

2.6. Ví dụ XSLT lọc và sắp xếp

Nhóm xây dựng sinh-vien.xsl để lấy các sinh viên từ 20 tuổi, sắp xếp tuổi giảm dần và tạo bảng HTML. Trích đoạn xử lý chính:

```
<xsl:for-each
  select="danhSachSinhVien/sinhVien[tuoi &gt;= 20]">
  <xsl:sort select="tuoi" data-type="number"
            order="descending"/>
  <tr>
    <td><xsl:value-of select="@id"/></td>
    <td><xsl:value-of select="hoTen"/></td>
    <td><xsl:value-of select="tuoi"/></td>
  </tr>
</xsl:for-each>
```

Với dữ liệu mẫu có tuổi 20, 19, 21, 20, bảng đầu ra gồm SV003, SV001 và SV004. SV002 bị loại bởi điều kiện lọc. Việc chỉ định data-type bằng number bảo đảm bước sắp xếp được thực hiện theo số trong ví dụ, thay vì dựa trên thứ tự ký tự của tuổi.

Kết quả HTML được sinh bởi chương trình kiểm tra riêng và lưu thành ket-qua-xslt.html. Trang demo đang dùng JavaScript để dựng bảng nên không cần thay đổi để chạy ví dụ này. Hai cách tiếp cận tạo đầu ra tương tự nhưng khác cơ chế: một cách áp dụng stylesheet, cách còn lại tạo các node DOM theo chương trình.

2.7. Phân tích biểu thức XPath bằng ví dụ

Nhóm sử dụng bốn sinh viên giống dữ liệu mẫu để xác định kết quả mong đợi trước khi thực thi. Điều này giúp phân biệt lỗi biểu thức, dữ liệu không khớp và sai kỳ vọng của người thao tác.

Biểu thức	Kết quả trên dữ liệu mẫu
/danhSachSinhVien/sinhVien/@id	SV001, SV002, SV003, SV004
/danhSachSinhVien/sinhVien[tuoi>=20]/hoTen	Dũng, Doanh và Tùng Dương
/danhSachSinhVien/sinhVien[tuoi>=21]/@id	SV003
count(/danhSachSinhVien/sinhVien)	4
/danhSachSinhVien/sinhVien[tuoi>100]/hoTen	Không có node phù hợp

Dấu / đầu tiên cho biết đường dẫn bắt đầu từ gốc tài liệu. Mỗi bước tiếp theo đi đến phần tử con có tên tương ứng; @id chọn thuộc tính thay vì phần tử. Cặp ngoặc vuông chứa điều kiện áp dụng lên các node sinhVien đã chọn. Nhờ đó, điều kiện tuổi được đánh giá trên từng bản ghi trước khi lấy trường tên.

Trong demo, kết quả đếm là số nên được đọc qua numberValue; kết quả tên là tập node nên được duyệt bằng iterateNext. Nếu chỉ dùng một cách đọc kết quả cho mọi

biểu thức, thao tác đếm và thao tác chọn tên sẽ không thể được xử lý thống nhất. Đây là lý do mã có các nhánh theo resultType.

2.8. XQuery tạo kết quả mới

Ví dụ sinh-vien.xq thể hiện cách xây dựng một tài liệu kết quả thay vì chỉ chọn nguyên các node tên. Đoạn truy vấn được viết cho bộ xử lý XQuery riêng và chưa được thực thi trong môi trường kiểm tra của báo cáo.

```
for $sv in doc("sinh-vien.xml")/danhSachSinhVien/sinhVien
where xs:integer($sv/tuoi) >= 20
order by xs:integer($sv/tuoi) descending, string($sv/@id)
return
  <ketQua ma="{data($sv/@id)}">
    {$sv/hoTen}
    {$sv/tuoi}
  </ketQua>
```

Trong ví dụ do nhóm thiết kế, mỗi sinh viên đáp ứng điều kiện được biến thành phần tử ketQua có thuộc tính ma và hai trường con. Truy vấn giả định tệp XML tồn tại ở vị trí bộ xử lý có thể đọc được và trường tuổi có thể ép thành số nguyên. Với một nguồn dữ liệu tùy ý, những giả định này cần được kiểm tra hoặc xử lý lỗi.

Kết quả dự kiến gồm ba phần tử cho SV003, SV001 và SV004. Thứ tự bổ sung theo id giúp ví dụ có tiêu chí rõ ràng khi hai sinh viên bằng tuổi. Không chạy trực tiếp biểu thức này trong ô XPath của ứng dụng, vì Document.evaluate không phải bộ xử lý XQuery.

PHẦN 3. JSON VÀ XỬ LÝ DỮ LIỆU TRONG JAVASCRIPT

3.1. Khái niệm và mô hình dữ liệu

JSON (JavaScript Object Notation) là định dạng văn bản dùng trao đổi dữ liệu, có nguồn gốc từ JavaScript nhưng độc lập với ngôn ngữ lập trình. JSON gồm object, array và các giá trị string, number, boolean, null. Tên thuộc tính trong object là chuỗi đặt trong dấu nháy kép. Mảng là một dãy giá trị có thứ tự. [2]

```
{
  "sinhVien": [
    {
      "id": "SV001",
      "hoTen": "Lường Tiến Dũng",
      "tuoi": 20,
      "email": "dung@example.com"
    }
  ]
}
```

So với XML ở phần trước, JSON biểu diễn danh sách bằng mảng và tuổi bằng giá trị số. JSON không có sẵn thuộc tính phần tử theo nghĩa XML; id được biểu diễn như các trường khác trong object. Khi ánh xạ giữa hai định dạng, ứng dụng phải quy định trường nào tương ứng với thuộc tính hoặc phần tử.

3.2. JSON.parse và JSON.stringify

JSON.parse() phân tích chuỗi JSON để trả về giá trị JavaScript. JSON.stringify() chuyển một giá trị JavaScript phù hợp sang chuỗi JSON. Đây là hai hướng xử lý dữ liệu khác nhau; chuỗi JSON và object JavaScript không phải cùng một đối tượng. [8][9]

```
const obj = JSON.parse(json);
const rows = obj.sinhVien;
const output = JSON.stringify({sinhVien: rows}, null, 2);
```

Tham số 2 ở cuối giúp thụt lề đầu ra cho dễ đọc. Trong ứng dụng, thao tác Parse & đọc JSON tạo dữ liệu cho bảng; thao tác xuất bảng tạo chuỗi JSON mới từ dữ liệu đã chỉnh sửa. JSON sai cú pháp, chẳng hạn dấu phẩy thừa ở cuối object, sẽ khiến parse phát sinh lỗi và được bắt trong khối try/catch.

3.3. Kiểm tra cấu trúc sau khi parse

Parse thành công chỉ cho biết chuỗi hợp lệ về cú pháp JSON. Ứng dụng vẫn cần kiểm tra sinhVien có phải mảng, mỗi phần tử có các trường cần thiết, tuổi có phải

số nguyên không âm và mã sinh viên có trùng nhau hay không. Ví dụ {"sinhVien":"abc"} là JSON đúng cú pháp nhưng không đúng mô hình của demo.

Nhóm không sử dụng JSON Schema trong phiên bản này. Các trường ngoài mô hình sinh viên có thể không được giữ lại khi chuyển đổi, vì đây là phép ánh xạ riêng cho ví dụ học tập, không phải công cụ chuyển đổi tổng quát.

3.4. Phân biệt chuỗi JSON và object trong ứng dụng

Khi người dùng gõ vào textarea, nội dung nhận được là chuỗi. Chương trình không thể coi chuỗi đó đã có trường sinhVien mà bỏ qua bước parse. Sau khi JSON.parse thành công, chương trình mới truy cập data.sinhVien, kiểm tra mảng và truyền từng bản ghi sang bảng. Đây là ranh giới giữa định dạng trao đổi và cấu trúc dữ liệu trong bộ nhớ.

```
const text = '{"sinhVien": [{"id": "SV001", "tuoi": 20}]}';
const data = JSON.parse(text);
console.log(typeof text);       // string
console.log(typeof data);       // object
console.log(data.sinhVien[0].tuoi); // 20
```

Đoạn ví dụ trên chỉ minh họa parse, không phải dữ liệu đầu vào đầy đủ của demo vì còn thiếu hoTen và email. Sự khác biệt này cũng cho thấy một đoạn mã có thể chạy đúng ở mức cú pháp nhưng vẫn không đáp ứng mô hình ứng dụng.

3.5. Các lỗi JSON thường gặp khi thao tác

Chuỗi nhập	Nhận xét
{"tuoi":20}	JSON hợp lệ nhưng chưa đủ mô hình demo
{'tuoi':20}	Sai vì dùng nháy đơn cho tên thuộc tính
{"tuoi":20,}	Sai vì dấu phẩy cuối object
{"tuoi":undefined}	undefined không phải giá trị JSON
{"tuoi":"20"}	JSON hợp lệ; tuổi đang là chuỗi

Chuỗi nhập	Nhận xét
{"tuoi":20.5}	JSON hợp lệ; không đạt tuổi nguyên của demo

Ví dụ tuổi "20" có thể nhìn giống số 20 trên màn hình, nhưng validate sử dụng Number.isSafeInteger nên không chấp nhận chuỗi. Việc không tự động ép mọi dữ liệu thành số giúp người trình bày minh họa rõ vai trò của kiểu giá trị. Ngược lại, ở XML, tuổi được đọc từ textContent nên cần một bước kiểm tra dạng văn bản rồi mới chuyển thành số.

JSON.parse không kiểm tra một chuỗi là tên người thật hay một số là tuổi hợp lý trong nghiệp vụ. Đó là trách nhiệm của mô hình và quy tắc ứng dụng. Trong phạm vi này, nhóm chỉ yêu cầu tuổi nguyên không âm; không đặt giới hạn tuổi tối đa tùy ý để làm thay đổi bài toán minh họa.

3.6. Sửa object và xuất lại dữ liệu

Khi đọc thành công, render tạo bản sao các object để lưu vào biến students. Người dùng sửa input tuổi thì bộ xử lý sự kiện cập nhật tuổi của bản ghi trong mảng này. Hai ô dữ liệu chưa đổi ngay; nút Xuất bảng là thời điểm người dùng yêu cầu tạo lại chuỗi từ dữ liệu trung gian.

```
const students = [
  {id: 'SV001', hoTen: 'Lường Tiến Dũng',
   tuoi: 20, email: 'dung@example.com'}
];
students[0].tuoi = 22;
const output = JSON.stringify({sinhVien: students}, null, 2);
```

Trong ví dụ, thao tác cập nhật thực hiện trên object trước; stringify chỉ tạo chuỗi phản ánh trạng thái mới. Việc ghi rõ bước xuất giúp tránh nhầm rằng textarea, bảng và object luôn tự đồng bộ sau mọi lần gõ.

PHẦN 4. SO SÁNH VÀ LỰA CHỌN XML, JSON

4.1. Đối chiếu theo yêu cầu

Tiêu chí	XML	JSON
Mô hình chính	Phần tử, thuộc tính, văn bản	Object, array và giá trị
Kiểu giá trị	Nội dung văn bản; schema có thể ràng buộc kiểu	Có sẵn number, boolean, string, null
Cách tổ chức	Cây phần tử; có namespace	Object và array lồng nhau
Kiểm tra cấu trúc	Có thể dùng DTD/XSD	Có thể dùng JSON Schema hoặc code
Truy vấn, biến đổi	XPath, XQuery, XSLT	Thường xử lý qua thư viện và ngôn ngữ lập trình
Với ví dụ sinh viên	Nhiều ký tự đánh dấu hơn	Cú pháp thường ngắn gọn hơn

Trong ví dụ, XML dùng cặp thẻ để bao nội dung, còn JSON dùng tên trường và dấu phân cách. Kích thước thực tế còn phụ thuộc tên trường, cách thụt lề, mã hóa và nén. Báo cáo không đưa ra số liệu tốc độ hoặc tỉ lệ tiết kiệm dung lượng vì chưa tiến hành phép đo tương ứng.

4.2. Khi nào dùng XML?

XML là lựa chọn phù hợp khi hệ thống hoặc tiêu chuẩn đã yêu cầu XML, khi cần namespace, mô hình tài liệu có cấu trúc hoặc quy trình kiểm tra và biến đổi sử dụng công nghệ XML. Lợi ích cần được đánh giá cùng chi phí công cụ và mức độ phức tạp của tài liệu.

4.3. Khi nào dùng JSON?

JSON phù hợp khi dữ liệu chủ yếu là object và array, khi các thành phần trao đổi đã hỗ trợ JSON và khi cần biểu diễn dữ liệu gần với cấu trúc xử lý của ứng dụng. Trong ví dụ Web bằng JavaScript, parse và stringify giúp triển khai minh họa ngắn gọn. Tuy vậy, JSON vẫn cần quy ước cấu trúc và kiểm tra dữ liệu.

4.4. Vì sao không chuyển đổi XML và JSON một cách máy móc?

Một tài liệu XML có thể chứa thuộc tính, nhiều phần tử cùng tên, namespace và văn bản xen kẽ phần tử con. Một object JSON không có ánh xạ duy nhất cho tất cả những cấu trúc đó. Nếu không xác định quy ước, cùng một XML có thể được biến thành nhiều JSON khác nhau nhưng đều có lý do thiết kế riêng.

Trong ví dụ sinh viên, nhóm đặt ra ánh xạ cụ thể: thuộc tính id đi vào trường id; ba phần tử hoTen, tuoi, email đi vào ba trường tương ứng; nhiều sinhVien được đặt trong một mảng. Phép chuyển đổi này đủ cho mô hình học tập, nhưng không giữ lại mọi thông tin hình thức của tài liệu nguồn như cách thụt lề hoặc comment.

Thành phần nguồn	Quy ước trong demo	Giới hạn
Thuộc tính id của sinhVien	Trường id trong object	Không phân biệt các thuộc tính ngoài mô hình
Nội dung tuoi của XML	Number trong JavaScript	Phải là số nguyên an toàn không âm
Nhiều phần tử sinhVien	Mảng sinhVien	Root phải đúng tên đã thống nhất
Trường JSON bổ sung	Không thuộc ánh xạ	Có thể mất khi đi qua XML
Khoảng trắng và comment XML	Không đưa vào dữ liệu trung gian	Không bảo toàn nguyên văn tài liệu

Do đó, kiểm tra vòng XML → JSON → XML nên đối chiếu dữ liệu được hỗ trợ, không so sánh hai chuỗi từng ký tự. Khác biệt cách xuống dòng không nhất thiết là mất dữ liệu; ngược lại, mất một trường ngoài mô hình là thay đổi nội dung cần được nhận biết.

4.5. Ba tình huống lựa chọn định dạng

Tình huống thứ nhất là trang Web nhỏ có dữ liệu gần với object và array. Nếu các bên liên quan đều sử dụng mô hình đó và đã thống nhất schema, JSON giúp biểu diễn trực tiếp. Mẫu sinh viên hiện tại thuộc tình huống này nên phần parse/stringify ngắn gọn và dễ trình diễn.

Tình huống thứ hai là tiếp nhận tài liệu đã được đối tác quy định bằng XML và XSD. Việc chuyển sang JSON chỉ vì nhìn gọn có thể làm mất lợi ích kiểm định hoặc đòi hỏi xây dựng quy ước mới. Khi đó, xử lý XML bằng công cụ phù hợp là lựa chọn cần được cân nhắc trước.

Tình huống thứ ba là cần tạo nhiều cách trình bày từ cùng nguồn XML. Stylesheet XSLT có thể tách quy tắc biến đổi khỏi dữ liệu. Tuy nhiên, nhóm vẫn phải xét năng lực công cụ, cách triển khai và người duy trì, không chỉ dựa vào khả năng về mặt cú pháp.

PHẦN 5. XÂY DỰNG ỨNG DỤNG MINH HỌA TRÊN TRÌNH DUYỆT

5.1. Mục tiêu và phạm vi

Ứng dụng cho phép người xem sửa XML và JSON trực tiếp, kiểm tra dữ liệu, hiển thị danh sách sinh viên và chạy XPath. Mã nguồn gồm index.html tạo giao diện, style.css định dạng và app.js xử lý tương tác. Có thể mở index.html trực tiếp hoặc chạy python -m http.server 8000 rồi truy cập http://localhost:8000.

Ứng dụng dùng HTML, CSS và JavaScript thuần; không cần thư viện bên ngoài. Dữ liệu nằm trong bộ nhớ trang và trở về mẫu khi tải lại. Các thông tin tuổi, email trong mẫu được đặt để minh họa. Việc này giúp demo chạy local và không phụ thuộc kết nối mạng.

5.2. Luồng xử lý

Thao tác	Xử lý chính	Kết quả
Kiểm tra & đọc XML	DOMParser, kiểm tra cấu trúc, render	Bảng dữ liệu và trạng thái
Parse & đọc JSON	JSON.parse, kiểm tra cấu trúc, render	Bảng dữ liệu và trạng thái
XML → JSON hoặc JSON → XML	Ánh xạ theo mô hình sinh viên	Cập nhật ô đích và bảng
Chạy XPath	Document.evaluate trên XML hiện tại	Danh sách tên, số hoặc giá trị khác
Sửa tuổi và xuất bảng	Kiểm tra bảng, tạo XML và stringify JSON	Cập nhật cả hai ô dữ liệu
Tạo lỗi thử	Nạp mẫu lỗi vào ô được chọn	Cho phép quan sát lỗi parser

Bảng chỉ cập nhật khi đọc thành công. Nếu lần đọc mới lỗi, bảng vẫn thể hiện lần đọc thành công gần nhất; thông báo cạnh ô dữ liệu cho biết lỗi mới. XPath đọc trực tiếp nội dung XML hiện tại nên không phụ thuộc lần cập nhật bảng.

5.3. Một số đoạn mã tiêu biểu

Phân tích XML và phát hiện lỗi parser:

```
const doc = new DOMParser().parseFromString(text, 'application/xml');
if (doc.getElementsByTagName('parsererror').length)
  throw Error('Sai cú pháp XML');
```

Chạy XPath bằng API trình duyệt:

```
const result = doc.evaluate(
  expression, doc, null, XPathResult.ANY_TYPE, null
);
```

Trong app.js, kết quả được phân loại theo number, string, boolean hoặc tập node để hiển thị phù hợp. Thông báo và giá trị người dùng được chèn bằng textContent; dữ liệu nhập không được ghép thành mã JavaScript để thực thi.

Khi xuất XML, các ký tự đặc biệt được escape trước khi ghép vào thẻ và thuộc tính. Việc chuyển đổi dùng các trường id, hoTen, tuoi, email đã thống nhất. Demo từ chối DOCTYPE và không triển khai phân tích DTD.

5.4. Mô hình dữ liệu trung gian và trách nhiệm của từng hàm

Hai định dạng cùng được đưa về mảng các object có id, hoTen, tuoi, email. Mảng này là điểm chung giữa thao tác đọc, hiển thị và xuất. Nhờ vậy, render không phải quan tâm dữ liệu ban đầu đến từ XML hay JSON; các khác biệt định dạng được xử lý trước khi render được gọi.

Hàm hoặc biến	Vai trò trong app.js
sample	Bộ dữ liệu minh họa để khôi phục
students	Mảng hiện đang được biểu diễn và sửa trên bảng
xmlDocument	Phân tích XML và phát hiện lỗi parser
parseXml, parseJson	Đọc từng định dạng về mô hình chung

Hàm hoặc biến	Vai trò trong app.js
validate	Kiểm tra kiểu giá trị, trường bắt buộc và id trùng
render	Tạo hàng bảng, input tuổi và sự kiện sửa
read	Điều phối đọc, chuyển đổi nếu cần và hiển thị trạng thái
toXml, escapeXml	Tạo chuỗi XML và bảo vệ ký tự đặc biệt
reset	Nạp lại dữ liệu, bảng và truy vấn mẫu

Một thay đổi quan trọng đối với mô hình, chẳng hạn thêm trường lop, cần được phản ánh ở tất cả các điểm: dữ liệu mẫu, parser, validate nếu trường bắt buộc, bảng hiển thị và bộ xuất. Chỉ thêm trường vào JSON không làm ứng dụng tự hiểu và bảo toàn trường đó. Bảng trách nhiệm giúp xác định phạm vi sửa khi mở rộng.

5.5. Phân tích quá trình kiểm tra tuổi và mã sinh viên

Ở phía XML, trước tiên mã lấy nội dung tuoi rồi trim khoảng trắng. Biểu thức chính quy chỉ chấp nhận các chữ số, nhờ đó chuỗi rỗng hoặc một phần số kèm chữ không bị chuyển thành giá trị hợp lệ ngoài ý muốn. Sau đó Number chuyển chuỗi sang số; validate tiếp tục yêu cầu số nguyên an toàn không âm.

Ở phía JSON, giá trị đã có kiểu sau parse nên validate kiểm tra trực tiếp. Ví dụ "20" là chuỗi và không được chấp nhận như 20. Khi người dùng sửa tuổi trên bảng, input trả chuỗi; chương trình có bước chuyển Number tương tự. Nếu input bị xóa hết, mã lưu NaN để lần xuất tiếp theo báo lỗi thay vì âm thầm biến chuỗi rỗng thành tuổi 0.

Với id, tập Set được khởi tạo rỗng trước khi duyệt các bản ghi. Mỗi mã được kiểm tra xem đã tồn tại hay chưa rồi mới thêm vào tập. Cách này diễn đạt trực tiếp yêu cầu không có hai sinh viên cùng mã. Demo so sánh mã theo chuỗi thực tế; nó không chuẩn hóa mọi khác biệt khoảng trắng hoặc chữ hoa, chữ thường của id.

5.6. Luồng lỗi và việc giữ lại kết quả cũ

Hàm read đặt các bước parse, kiểm tra và render trong try/catch. Khi một bước ném lỗi trước render, bảng chưa bị thay thế nên còn dữ liệu của lần đọc thành công trước đó. Điều này hữu ích khi trình diễn vì người xem vẫn có kết quả đối chiếu. Tuy nhiên, trạng thái cạnh ô dữ liệu phải được quan sát cùng với bảng để không nhầm rằng bản nhập lỗi vừa được chấp nhận.

```
Nhấn nút đọc
  -> Parse nội dung
  -> Kiểm tra mô hình
  -> Nếu đạt: thay bảng, cập nhật nguồn và trạng thái
  -> Nếu lỗi: hiện thông báo, giữ bảng cũ
```

Các nút tạo lỗi nạp mẫu lỗi mới vào ô tương ứng; chúng không cố sửa một ký tự bất kỳ của nội dung tùy ý mà người dùng đang viết. Nút Khôi phục mẫu khôi phục cả hai ô và bảng. Các thao tác này thay thế dữ liệu đang nhập bằng dữ liệu mẫu.

5.7. Tạo giao diện bằng DOM và xử lý dữ liệu nhập

render tạo từng tr, td và input bằng document.createElement rồi gán chuỗi vào textContent. Nếu tên sinh viên chứa một đoạn giống thẻ HTML, đoạn đó được hiển thị như dữ liệu thay vì được dùng để tạo thêm cấu trúc giao diện. Đây là lựa chọn phù hợp cho bảng cần hiển thị văn bản từ nội dung người dùng.

Tại bước xuất XML, escapeXml xử lý các ký tự có ý nghĩa trong đánh dấu trước khi ghép chuỗi. Hai kỹ thuật textContent và escape XML giải quyết hai vị trí khác nhau: một bên là hiển thị trong HTML, một bên là tạo tài liệu XML mới. Không nên bỏ một bước chỉ vì đã có bước còn lại.

Demo không đặt ra khẳng định bảo mật tổng quát. Ví dụ, chưa có giới hạn dung lượng tài liệu hoặc cơ chế đọc XML theo luồng nên tài liệu quá lớn có thể làm trải nghiệm chậm. Đây là công cụ học tập xử lý nội dung cục bộ, không phải dịch vụ nhận tài liệu không giới hạn từ nhiều người dùng.

5.8. Ma trận kiểm thử chức năng trình duyệt

Đã kiểm tra cú pháp JavaScript bằng node --check app.js. Mã nguồn giải nén từ ZIP nộp được kiểm tra trực tiếp trên Edge: 17 kiểm tra tự động đều đạt. Bảng sau liệt kê các tình huống chức năng và kết quả mong đợi đã được đối chiếu. Kết quả áp dụng cho bộ tình huống đã chạy, chưa bao phủ mọi đầu vào và môi trường.

Mã ca	Thao tác	Kết quả mong đợi
WEB01	Đọc XML mẫu	Bảng đủ 4 sinh viên, nguồn XML
WEB02	Đọc JSON mẫu	Bảng cùng dữ liệu, nguồn JSON
WEB03	Tạo lỗi XML rồi đọc	Báo lỗi parser, bảng cũ không bị xóa
WEB04	Tạo lỗi JSON rồi đọc	Báo lỗi cú pháp JSON
WEB05	Đổi tuổi JSON thành chuỗi	Báo tuổi không đạt kiểu số nguyên
WEB06	Sửa tuổi thành số âm	Khi xuất, báo lỗi giá trị
WEB07	Xóa giá trị input tuổi	Khi xuất, báo lỗi; không tự đổi thành 0
WEB08	Hai bản ghi có cùng id	Báo mã trùng
WEB09	Xuất bảng sau khi sửa tuổi	Cả XML và JSON chứa tuổi mới
WEB10	XPath tuổi từ 20	Có ba tên theo dữ liệu mẫu
WEB11	XPath không khớp node	Hiện không có node phù hợp
WEB12	Gõ XPath sai cú pháp	Hiện thông báo lỗi truy vấn
WEB13	JSON có mảng sinhVien rỗng	Đọc thành công, bảng có 0 sinh viên

Mã ca	Thao tác	Kết quả mong đợi
WEB14	Tên chứa dấu & rồi chuyển đổi	Đọc lại đúng ký tự, XML đã escape
WEB15	Khôi phục mẫu	Hai ô, bảng và XPath trở về mẫu

Khi thực hiện WEB09, cần bấm đọc lại từng ô sau khi xuất để kiểm tra dữ liệu mới thực sự parse được, thay vì chỉ nhìn chuỗi đầu ra. Khi thực hiện WEB14, cần kiểm tra nội dung qua vòng chuyển đổi để phát hiện lỗi escape hoặc mất dữ liệu. Các ca này nhắm vào hành vi có ý nghĩa thay vì chỉ xác nhận nút bấm có tồn tại.

5.9. Kiểm tra bổ sung cho các ví dụ công nghệ XML

Nhóm bổ sung bộ ví dụ chạy ngoài trình duyệt trong report/examples. Chương trình check_examples.py dùng lxml để kiểm tra DTD, XSD, XPath và XSLT. Những kết quả này xác nhận các ví dụ tương ứng trong báo cáo; không thay thế kiểm thử giao diện hoặc biến trang demo thành bộ kiểm định XSD.

Nhóm tình huống đã chạy	Số ca	Kết quả
Tài liệu mẫu qua DTD và XSD	2	Đạt
Tuổi chữ: DTD chấp nhận, XSD từ chối	2	Đạt
XSD từ chối tuổi âm	1	Đạt
Thiếu email bị DTD/XSD từ chối	2	Đạt
Mã trùng bị DTD/XSD từ chối	2	Đạt

Nhóm tình huống đã chạy	Số ca	Kết quả
XPath đếm và lọc theo tuổi	3	Đạt
XSLT lọc và sắp xếp thành bảng	1	Đạt

Tổng cộng 13 kiểm tra đã chạy thành công với lxml 6.1.3. Kết quả lưu trong ket-qua-kiem-tra.json; HTML của phép biến đổi lưu trong ket-qua-xslt.html. XQuery chỉ được cung cấp ở dạng ví dụ nguồn và chưa chạy vì bộ kiểm tra này không có bộ xử lý XQuery.

Để chạy lại trong môi trường Python đã có lxml, sử dụng lệnh python report/examples/check_examples.py tại thư mục XML-JSON-Demo sau khi giải nén mã nguồn. Cần giữ các tệp XML, DTD, XSD và XSL cùng thư mục examples như cấu trúc được cung cấp.

5.10. Đánh giá kết quả và hướng hoàn thiện

Kết quả hiện tại đáp ứng mục tiêu minh họa cùng một tập sinh viên qua hai định dạng và cho phép người xem tạo thay đổi có thể quan sát được. Tách dữ liệu trung gian giúp các thao tác đọc và xuất dùng cùng mô hình. Các ví dụ bổ sung làm rõ khác biệt giữa cú pháp, kiểm định cấu trúc, truy vấn và biến đổi tài liệu.

Hạn chế chính là mô hình cố định và dữ liệu không lưu qua lần tải lại. Bộ kiểm tra hiện tại đã bao phủ các thao tác minh họa chính nhưng chưa đánh giá trên nhiều trình duyệt, tài liệu rất lớn hoặc thiết bị di động. Có thể mở rộng kiểm thử, bổ sung hướng dẫn nhập, sau đó cân nhắc lưu cục bộ hoặc tích hợp kiểm định schema. Nếu thêm API hoặc cơ sở dữ liệu, cần xác định lại yêu cầu nghiệp vụ thay vì chỉ gắn thêm công nghệ cho đủ tên.

KẾT LUẬN

Báo cáo trình bày hai cách biểu diễn dữ liệu XML và JSON, phân biệt cú pháp với cấu trúc dữ liệu theo yêu cầu, đồng thời giới thiệu các công nghệ liên quan đến XML và thao tác xử lý JSON trong JavaScript. Phần so sánh cho thấy việc lựa chọn định dạng cần dựa vào yêu cầu và hệ công cụ của hệ thống.

Ứng dụng minh họa gắn kiến thức với thao tác: đọc tài liệu thành bảng, chủ động tạo lỗi, truy vấn XPath và xuất lại dữ liệu sau khi sửa. Giới hạn hiện tại là chỉ hỗ trợ mô hình sinh viên, chưa có kiểm định DTD/XSD, chưa thực thi XSLT/XQuery và chưa lưu dữ liệu lâu dài. Các hướng mở rộng có thể gồm bổ sung schema thực, stylesheet biến đổi và mở rộng kiểm thử trên nhiều trình duyệt.

TÀI LIỆU THAM KHẢO

[1] W3C. Extensible Markup Language (XML) 1.0, Fifth Edition. https://www.w3.org/TR/xml/

[2] IETF. RFC 8259 — The JavaScript Object Notation (JSON) Data Interchange Format. https://www.rfc-editor.org/rfc/rfc8259

[3] W3C. XML Schema Part 0: Primer, Second Edition. https://www.w3.org/TR/xmlschema-0/

[4] W3C. XSL Transformations (XSLT), Version 1.0. https://www.w3.org/TR/xslt-10/

[5] W3C. XML Path Language (XPath), Version 1.0. https://www.w3.org/TR/xpath-10/

[6] MDN Web Docs. Document: evaluate() method. https://developer.mozilla.org/en-US/docs/Web/API/Document/evaluate

[7] W3C. XQuery 3.1: An XML Query Language. https://www.w3.org/TR/xquery-31/

[8] MDN Web Docs. JSON.parse(). https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/parse

[9] MDN Web Docs. JSON.stringify(). https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

[10] Nhóm Một anh cả ngã về M0. Slide XML & JSON (Slide-XML-JSON.pdf) và mã nguồn demo index.html, style.css, app.js trong Ma-nguon-demo.zip.

PHỤ LỤC. BỘ VÍ DỤ VÀ HƯỚNG DẪN TÁI LẬP

A. Các tệp bàn giao

Bộ nộp gồm ba tệp: Bao-cao-XML-JSON-MAT3385.pdf (báo cáo), Slide-XML-JSON.pdf (slide thuyết trình) và Ma-nguon-demo.zip (mã nguồn cùng bộ ví dụ). Giải nén Ma-nguon-demo.zip được thư mục XML-JSON-Demo. Trong đó, index.html, style.css và app.js là mã ứng dụng; README.md hướng dẫn chạy. Các ví dụ công nghệ XML nằm trong report/examples theo bảng sau:

Tệp	Vai trò
sinh-vien.xml	Dữ liệu nguồn của bốn sinh viên
sinh-vien.dtd	Định nghĩa cấu trúc và thuộc tính ID
sinh-vien.xsd	Kiểu dữ liệu và ràng buộc mã duy nhất
sinh-vien.xsl	Lọc tuổi, sắp xếp và xuất bảng HTML
sinh-vien.xq	Ví dụ XQuery, chưa thực thi
check_examples.py	Chạy các kiểm tra DTD/XSD/XPath/XSLT
ket-qua-kiem-tra.json	Kết quả 13 ca kiểm tra đã chạy
ket-qua-xslt.html	HTML đầu ra của ví dụ XSLT

B. Chạy trang demo

Mở index.html bằng trình duyệt để dùng ngay. Nếu cần địa chỉ localhost, mở terminal tại thư mục gốc và chạy python -m http.server 8000, sau đó truy cập http://localhost:8000. Không cần backend hoặc tài khoản. Nhấn Ctrl+C ở terminal để dừng máy chủ tĩnh.